Computer Vision

Nicola Conci
a.y. 2025/2026
University of Trento

Davide Donà

June 2026

Notes Repository

Computer Vision is the science and technology of machines that **see**, where "see" means being able to extract information from an image, necessary to perform a task.

Contents

1 Images and Videos

Before introducing the concepts of images and videos, we first need to define the concept of a **signal**, how to represent it, and how to process it.

1.1 The Processing Chain

The **processing chain** of an image starts in the real world with a physical signal we want to capture. This signal goes through the following steps:

- **Acquisition**: The process of transforming a real-world physical signal into a digital one by means of a **sensor**. In this process, we essentially project a portion of the real 3D world onto a 2D plane. This process is inherently affected by **noise** from the sensor, the environment, etc., which can be modeled as an additive component to the signal.
- **Enhancement**: The process of applying transformations or **filters** to the digital signal to make the picture more suitable for our specific goals (e.g., noise reduction, contrast enhancement).
- **High-level Processing**: The extraction of useful information from the signal. This is where Computer Vision algorithms come into play to apply intelligent processing (e.g., object detection, tracking, segmentation).

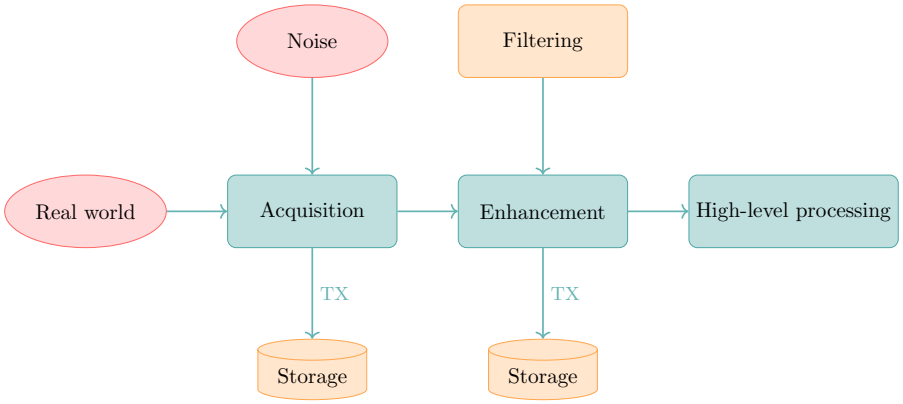


Figure 1: Processing chain of an image.

1.2 Acquisition and Digitization

To process an image on a computer, the continuous physical signal must be converted into a discrete digital format. When talking about images, the sensor is typically a camera that captures light intensity, wavelength (color), or

temperature. Digitalization requires two fundamental steps: **sampling** and **quantization**.

1.2.1 Sampling

To transform a real-world continuous signal into a digital one, we must sample it. For video signals, sampling occurs across three dimensions: the two spatial dimensions (x and y) and the temporal dimension (t).

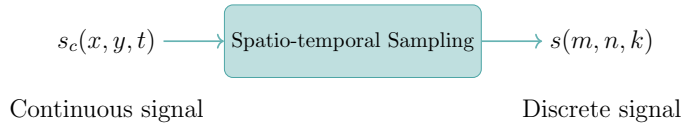


Figure 2: Spatio-temporal sampling of a video signal.

This process is called **spatio-temporal sampling**, transforming a continuous signal $s_c(x, y, t)$ into a discrete signal $s(m, n, k)$, where m, n, and k are the indices of the spatial and temporal dimensions.

1.2.2 Quantization

Once we have a discrete spatial signal, we must quantize it, meaning we assign a finite number of values to represent the light intensity at each sampled point.

The most common quantization uses 8 bits per pixel (bpp), which allows for 256 different values per channel. This is an excellent compromise between image quality and memory storage (exactly one byte per pixel channel). Using fewer than 8 bpp often results in a loss of quality, introducing a visual artifact known as **contouring**.

After quantization, pixel values are typically represented as integers in the range $[0, 255]$, where 0 represents no intensity (black) and 255 represents maximum intensity.

1.2.3 Digital Images

The output of the sampling and quantization processes is a 2D matrix of **pixels**. A digital image is essentially a collection of 2D coordinates, where each pixel represents the projection of a real-world point and the quantized intensity of the light that hit the sensor at that location.

Pixels are numbered starting from the top-left corner. The value of a pixel at position (c, r) for a given image I is denoted as $I(c, r)$, where c is the column index and r is the row index. For grayscale images, a pixel carries a single value. For color images, it carries a tuple of values (e.g., Red, Green, Blue).

1.3 Color Representation

Color is a perceptual attribute that humans associate with objects, mathematically defined as a combination of different **wavelengths** of light.

1.3.1 Acquiring Color: The Bayer Pattern

To create a color image, a camera sensor must capture light information for multiple color components. The perfect physical solution would be using three separate sensors (one for red, green, and blue), but this is expensive and impractical.

Instead, cameras use a **Bayer pattern**, a mosaic of color filters placed directly over a single sensor. The filters are arranged to mimic the human eye's natural sensitivities:

- **50% Green**: To capture luminance information, to which the human eye is most sensitive.
- **25% Red**: To capture red wavelength information.
- **25% Blue**: To capture blue wavelength information.

This pattern allows the camera to capture color in a single shot, interpolating the missing colors for each pixel afterward.

1.3.2 The RGB Color Space

The most common digital representation of color is **RGB** (Red, Green, Blue). This is an *additive color model*: starting from a black image (no light), we add different intensities of red, green, and blue to create a wide spectrum of colors.

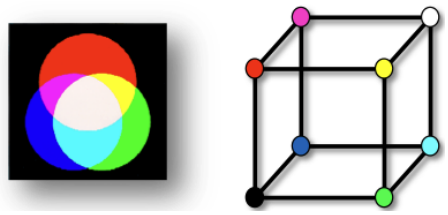


Figure 3: Additive color model.

Note: The opposite approach is the *subtractive color model* (CMY/CMYK), where we start with white (like a piece of paper) and subtract light by applying cyan, magenta, and yellow pigments.

Limitations of RGB While intuitive for hardware (like monitors), RGB is not optimal for image processing. The human eye is vastly more sensitive to luminance (brightness) than to chrominance (color variations).

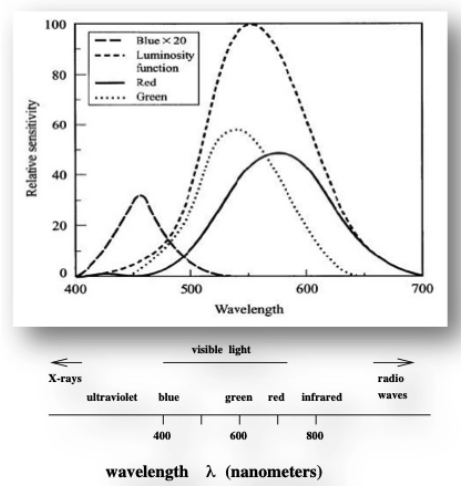


Figure 4: Sensitivity of the human eye to different colors.

Because RGB mixes luminance and chrominance across all three channels, the R, G, and B components are highly correlated. This redundancy makes RGB inefficient for tasks like compression or human-centric image processing.

1.3.3 Alternative Color Spaces

YCbCr (Used for Compression) The **YCbCr** space separates luminance (Y) from color information (Cb and Cr). Because humans are less sensitive to color detail, we can heavily compress (down-sample) the Cb and Cr channels without noticeably degrading the image quality. This principle is the backbone of JPEG and MPEG compression standards.

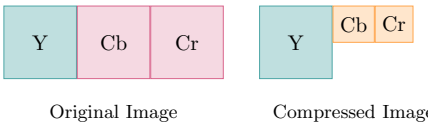


Figure 5: YCbCr compression example.

As shown above, by down-sampling the Cb and Cr components by a factor of 2, we halve the data size with almost zero perceived loss in quality.

HSV (Used for Processing and Graphics) The **HSV** (Hue, Saturation, Value) model separates the base color (Hue) from its intensity (Value) and its purity/vibrancy (Saturation). This mimics how humans intuitively describe color, making it highly effective for computer graphics, artistic editing, and computer vision tasks like color-based object tracking.

1.4 Videos

While static images provide information about static scenes, they lack motion information such as temporal evolution and dynamic interactions. Videos capture a sequence of images over time, providing a richer representation of the world.

A video can be seen as a sequence of 2D images, that represent the projection of a **moving** 3D scene onto the video camera sensor. Usually videos have a lower resolution than images, as they would be too heavy to store and transmit otherwise. Each second of video is composed by a certain number of frames (typically 24, 30 or 60), and each frame is a 2D image. For videos lasting several minutes, the amount of data can be huge. Also, the information carried by videos is mostly given by the motion, and not by the color information.

2 Image Elaboration and Feature Extraction

Once images are acquired and digitized, we can extract important features from them. Usually, we are interested in two primary characteristics:

- **Color** information and its distribution, described by the image's histogram.
- **Shape** information, given by contours and edges of subjects in the image.

When dealing with videos, we are also interested in how these features evolve over time. This introduces several challenges:

- **Consistency**: The features of the same object should remain consistent over time, even if the object moves or slightly changes appearance.
- **Evolution**: An object's features can change due to displacements, occlusions, or lighting changes.
- **Scene evolution**: The overall scene can change as new objects appear and old ones disappear.

2.1 Color Analysis: The Histogram

A histogram is a simple way to describe the color or intensity distribution of an image. It acts as a probability distribution of the pixel values. For a grayscale image, the histogram is a 1D array of 256 values, where each bin counts the number of pixels having that specific intensity.

For an $M \times N$ grayscale image I:

- The sum of all histogram bins equals the total number of pixels:

$$\sum_{i=0}^{255} H(i) = M \times N$$

- We can normalize the histogram to represent a probability distribution:

$$hist(i) = \frac{\text{count}(I(c,r) = i)}{M \times N}$$

This concept can be extended to color images by computing a separate histogram for each channel (e.g., R, G, B).

2.1.1 Information from the Histogram

The histogram acts as a global signature for the image, providing insights such as:

- The **mean intensity**, indicating if the image is generally bright, dark, or well-exposed.
- The **contrast distribution**, showing whether pixel values are clustered together or spread across the spectrum.

2.1.2 Histogram Operations

We can manipulate the histogram to enhance the image visually. Various techniques include:

Stretching (Contrast Enhancement)

Stretching improves contrast by expanding the histogram to cover the entire $[0, 255]$ range using a piece-wise linear transformation. Given two thresholds, t_1 and t_2, the transformation from the original pixel intensity $I(c, r)$ to the enhanced intensity $I'(c, r)$ is defined by the following function:

$$I'(c,r) = \begin{cases} 0 & \text{if } I(c,r) < t_1 \\ \frac{255}{t_2 - t_1} \cdot (I(c,r) - t_1) & \text{if } t_1 \leq I(c,r) \leq t_2 \\ 255 & \text{if } I(c,r) > t_2 \end{cases}$$

The resulting function is piece-wise linear, featuring three distinct segments: a flat segment clamped to 0, a linear mapping segment that scales the intermediate values across $[0, 255]$, and a final flat segment clamped to 255.

Equalization

Ideally, a high-contrast image has a relatively flat histogram where all intensities are equally represented. We can approximate this via **equalization**:

- **Cumulative Histogram**: First, compute the cumulative distribution:

$$CHist_I(i) = \sum_{k=0}^{i} hist(k)$$

- **Transformation**: Then, map the original pixels to new values:

$$hist_{equalized}(i) = \frac{CHist_I(i) - CHist_I(0)}{M \times N} \times 255$$

2.2 Spatial Filtering and Convolution

To extract shape information or remove noise, we manipulate groups of neighboring pixels. The fundamental mathematical operation used to apply these neighborhood filters is **convolution**.

2.2.1 The Convolution Operation

To apply a filter (often called a kernel or mask) to an image, we perform a 2D convolution between the image f and the filter h:

$$g(x,y) = (f * h)(x,y) = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} f(i,j)h(x-i, y-j) \tag{2.1}$$

In practical terms, we flip the filter h 180 degrees (equivalent to flipping horizontally and vertically), slide it over the image f, and compute the sum of element-wise products at each position.

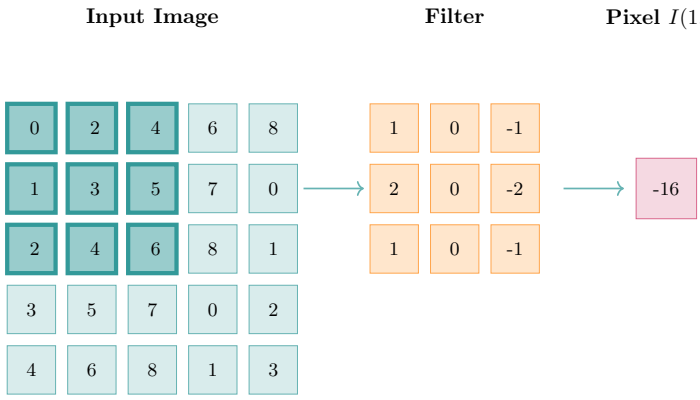


Figure 6: Example of convolution between an image and a filter.

Note: The output must be stored in a separate, new image array to ensure we process subsequent pixels using the original, unmodified neighboring values.

Handling image borders

When the filter overlaps the borders of the image, we cannot compute the convolution as usual, since some neighboring pixels are missing. Common strategies to handle this include:

- **Zero-padding**: Assume missing pixels are 0.

- **Replication**: Replicate the nearest border pixel values.
- **Reflection**: Mirror the image across the border.

2.2.2 Smoothing and Enhancement (Low-pass Filters)

Smoothing removes high-frequency components (like sharp edges and noise), leaving behind low-frequency components (smooth regions). We will now discuss three common low-pass filters: the average filter, the Gaussian filter, and the median filter.

Average Filter

A simple average filter uses a kernel of size $k \times k$ (e.g., 5×5) where each element is $\frac{1}{k^2}$. The output pixel value is the average of the neighboring pixels. For a 5×5 kernel, the output pixel at position (c, r) is computed as:

$$I'(c,r) = \frac{1}{25} \sum_{i=-2}^{2} \sum_{j=-2}^{2} I(c+i, r+j) \tag{2.2}$$

The larger the kernel, the stronger the blur.

Gaussian Filter

A Gaussian filter is a weighted low-pass filter that uses a kernel defined by the two-dimensional Gaussian function, which is typically centered at the origin of the kernel:

$$G(x,y) = ce^{-\frac{x^2+y^2}{2\sigma^2}} \tag{2.3}$$

where x and y represent the displacement from the center of the kernel. The variable σ controls the spread of the Gaussian, and c is a normalization constant ensuring that the sum of all kernel values equals 1.

Unlike a simple average filter, it assigns higher importance to pixels that are closer to the center of the kernel, resulting in a more natural and visually pleasing blur. It is **isotropic** (circularly symmetric), meaning it smooths equally in all directions. The standard deviation σ controls the blur intensity.

Median Filter

Unlike convolution-based filters, the median filter is non-linear. It sorts the neighborhood pixels by value and outputs the statistical median. It is highly robust to outliers and is particularly effective at removing "salt-and-pepper" (spike) noise without heavily blurring edges.

2.2.3 Edge Detection (High-Pass Filtering)

Edges represent abrupt spatial variations in image intensity and are fundamental features for image analysis. Detecting edges requires estimating local intensity variations, which can be achieved through image derivatives.

Gradient-Based Edge Detection

Since images are defined over a two-dimensional domain, intensity changes must be analyzed along both the horizontal (x) and vertical (y) directions. Let $f(x, y)$ denote the image intensity function. The rate of change along a generic direction r, identified by an angle θ, is given by the directional derivative:

$$\frac{\partial f}{\partial r} = f_x \cos\theta + f_y \sin\theta \tag{2.4}$$

where f_x and f_y denote the partial derivatives of the image intensity with respect to x and y, respectively.

The **direction** of maximum intensity variation corresponds to the gradient orientation, obtained by maximizing the directional derivative with respect to θ:

$$\theta_g = \tan^{-1}\left(\frac{f_y}{f_x}\right) \tag{2.5}$$

The **strength** of the edge is quantified by the gradient magnitude:

$$\|\nabla f\| = \sqrt{f_x^2 + f_y^2} \tag{2.6}$$

Discrete Gradient Approximation: Sobel Filters

In practical image processing applications, continuous derivatives are approximated using discrete convolution kernels. A common choice is the **Sobel operator**, which estimates image gradients along the horizontal and vertical directions through finite impulse response (FIR) filters:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \qquad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \tag{2.7}$$

After convolving the image with these kernels, the horizontal and vertical responses, denoted as D_x and D_y, are combined to estimate the overall edge intensity:

$$D = \sqrt{D_x^2 + D_y^2} \tag{2.8}$$

Binarization

Once we compute the continuous edge strengths, we apply a threshold to create a binary image: pixels above the threshold are classified as "edge" (1), and those below as "background" (0).

2.3 Morphological Filters

Morphology operators are non-linear transformations typically applied to **binary images** (like those produced after edge binarization). They use a **Binary Structuring Element** (BSE) to analyze, extract, or fit shapes within the image.

2.3.1 Fundamental Operations

- **Dilation**: The structuring element S is swept over the binary image B. If the origin of S touches a foreground pixel (1), all pixels covered by S are set to 1. This expands shapes and bridges small gaps.

$$B \oplus S = \bigcup_{b \in B} S_b \tag{2.9}$$

- **Erosion**: The structuring element S is swept over B. A pixel remains 1 only if the *entire* structuring element fits completely inside the foreground. This shrinks shapes and removes small isolated noise.

2.3.2 Compound Operations

Opening and closing are combinations designed to clean up shapes without drastically changing their overall area:

- **Opening (Erosion followed by Dilation)**: Removes small objects or thin protrusions from the foreground while preserving the size of larger shapes.
- **Closing (Dilation followed by Erosion)**: Fills small holes and smooths inward-curving boundaries without expanding the overall shape.

3 Models

To represent the real world using equations, we first need to define a model. A model is a **mathematical approximation** of the real world, which allows describing events and phenomena using parametric representations. The parameters of the model can be used for processing purposes.

3.1 Pinhole Camera Model

The first model we will discuss is the **pinhole camera model**, which relates to how a camera captures an image. It assumes that an object in the real world emits light rays that pass through a small hole (the pinhole) and project onto an image plane, creating an image.

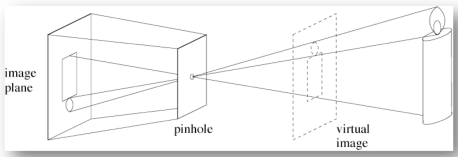


Figure 7: Pinhole Camera Model

The pinhole camera model is a simple geometric model that captures the essential features of image formation.

We want to make sure that only a **single ray of light** from each point of the object passes through the pinhole (ideally, the pinhole should be infinitely small), which allows us to create a clear image on the image plane. Bigger holes would allow multiple rays from the same point to pass through, resulting in a blurry / out-of-focus image.

On the image plane, the rays of light from the object are projected, creating an image which is flipped upside down. Its size is determined by the distance between the object and the pinhole, and the distance between the pinhole and the image plane (also known as the **focal length** f). The relationship between the object size, the image size, and the distances can be described using similar triangles:

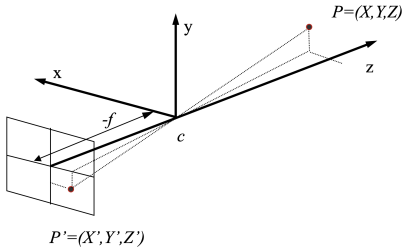


Figure 8: Projection of the point P onto the image plane

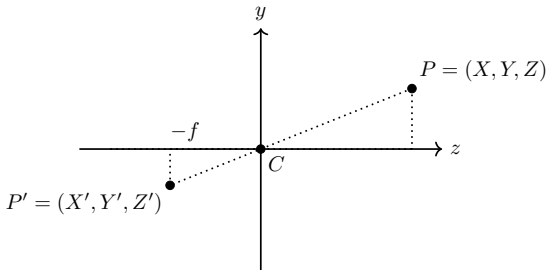


Figure 9: Projection of the point P onto the image plane (2D representation)

As shown in the figure, the point P in the real world is projected onto the image plane as P'. The coordinates of P' can be obtained using the geometric properties of the pinhole camera model. The triangles formed by the points P, C (the pinhole), and P' are similar, which means that the ratios between the corresponding sides are equal.

This leads to the following relations:

$$\begin{cases} Z' = -f \\ \dfrac{Z}{Z'} = \dfrac{X}{X'} \\ \dfrac{Z}{Z'} = \dfrac{Y}{Y'} \end{cases}$$

Substituting $Z' = -f$ and rearranging the equations, we obtain:

$$X' = -f\frac{X}{Z}$$
$$Y' = -f\frac{Y}{Z}$$

Therefore, the projection of the 3D point $P = (X, Y, Z)$ onto the image plane is given by:

$$P' = \left(-f\frac{X}{Z},\ -f\frac{Y}{Z},\ -f\right)$$

3.1.1 Features of the model

The pinhole camera model has several important features:

- Further objects appear smaller in the image, which is a consequence of the projection process. It is not able to correctly capture depth information directly.
- Parallel lines in the real world appear to converge towards a single point in the image, known as the **vanishing point**. The line connecting the vanishing points is called the **horizon line**. Vertical lines in the real world will still appear vertical in the image, being perpendicular to the horizon line.
- The image is inverted (upside down) due to the way light rays pass through the pinhole.
- The model assumes that the pinhole is infinitely small, which is an idealization. This would require an infinite amount of light to pass through to create a visible image, which is not practical.

3.2 Lenses

Up until now, we have assumed that our cameras were pinhole cameras, which is not the case in the real world. In reality, cameras use **lenses** to focus light onto the image plane. This allows a wider set of light rays from the same point P to merge at point P' on the image plane, creating a brighter image without losing sharpness—provided the distances obey the *thin lens equation*.

3.2.1 Thin Lens Equation

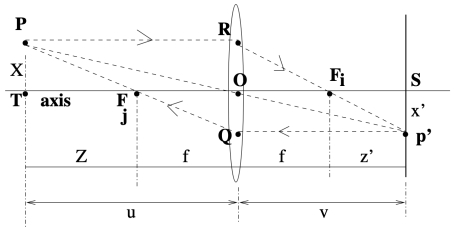


Figure 10: Thin Lens model.

Suppose that the origin of the coordinate system is at the lens center C, that we have an object at distance u from the lens, and an image plane at distance v from the lens. Let X be the object height and X' be the projected image height. By considering the similar triangles formed by rays passing through the optical center, we have:

$$\frac{X}{u} = \frac{X'}{v} \tag{3.1}$$

Let Z be the distance from the object to the object-side focal point, and Z' be the distance from the image-side focal point to the image plane. We can then rewrite u as $u = Z+f$, and v as $v = Z'+f$. Substituting these into the equation gives:

$$\frac{X}{Z+f} = \frac{X'}{Z'+f} \tag{3.2}$$

By considering the similar triangles formed by the ray parallel to the optical axis (which subsequently passes through the focal point on the image side), we obtain the relation $\frac{X}{f} = \frac{X'}{Z'}$, which can be rearranged as:

$$X = \frac{fX'}{Z'}$$

Substituting X back into our earlier equation:

$$\frac{fX'}{Z'(Z+f)} = \frac{X'}{Z'+f}$$

We can cancel out the X' terms and rearrange the equation to get:

$$\begin{aligned} f(Z'+f) &= Z'(Z+f) \\ fZ' + f^2 &= Z'Z + fZ' \end{aligned}$$

Subtracting fZ' from both sides yields Newton's lens equation:

$$ZZ' = f^2 \tag{3.3}$$

Since $Z = u - f$ and $Z' = v - f$, we can substitute these back into equation 3.3:

$$\begin{aligned} (u-f)(v-f) &= f^2 \\ uv - uf - vf + f^2 &= f^2 \\ uv &= f(u+v) \end{aligned}$$

Dividing both sides by uvf, we derive the **thin lens equation**:

$$\frac{1}{f} = \frac{1}{u} + \frac{1}{v} \tag{3.4}$$

This equation describes the relationship between the focal length f, the object distance u, and the image plane distance v. If the object goes to infinity ($u \to \infty$), the image plane must be located exactly at the focal length ($v = f$).

3.2.2 Focus & Depth of Field

The set of parameters (u, v) that satisfy the thin lens equation allows us to focus on objects at different distances. If we move the image plane (changing v), the point P' will project to a different area, meaning the objects will be out of focus. Similarly, if P moves (changing u), the image plane must be adjusted to keep the object in focus.

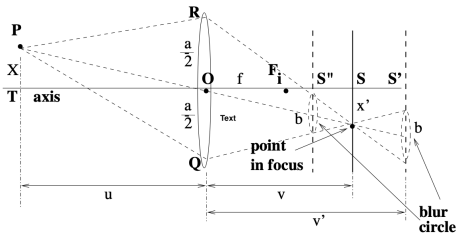


Figure 11: Focus and out-of-focus objects.

When out of focus, instead of a single point P', we get a **blur circle** of diameter b.

Obviously, since we are usually not working with a single point, but with an extended object, we cannot have all points in focus at the same time. We must define a priori an acceptable level of blur. Using the thin lens equation, we can determine the range of distances where objects remain acceptably sharp, known as the **depth of field**.

Let a be the lens aperture diameter (the diameter of the lens). The range of acceptable image plane distances is:

$$\begin{cases} v' = \frac{a+b}{a} v \\ v'' = \frac{a-b}{a} v \end{cases}$$

Translating this to object distances gives us the near limit (u_n) and far limit (u_r) of the depth of field:

$$\begin{cases} u_n = \frac{u(a+b)}{a+\frac{bu}{f}} \\ u_r = \frac{u(a-b)}{a-\frac{bu}{f}} \end{cases}$$

Notice that:

- If f becomes smaller, u_r becomes larger, resulting in a wider depth of field.

- If $u \to \infty$, the rays are parallel, $v = f$, and all distant objects remain in focus.

3.2.3 Autofocus

Modern cameras use autofocus systems to adjust the image plane position:

- **Active autofocus**: The camera emits a signal (e.g., infrared) and measures the time of flight to estimate distance. It struggles with distant objects, interference, or highly reflective surfaces.
- **Passive autofocus**: The camera analyzes the image itself (often targeting high-contrast edges) to maximize sharpness.

3.2.4 Resolution, Blur, and Resolving Power

A camera with an $N \times M$ pixel sensor can detect at most $\frac{N}{2}$ horizontal lines (one pixel must be left between two lines to distinguish them). We define the **resolving power** R_p as the maximum number of lines per unit distance the camera can distinguish:

$$R_p = \frac{1}{2\Delta} \tag{3.5}$$

where Δ is the spacing between two distinguishable lines on the sensor.

Example: Consider a 36mm $\times$ 24mm sensor with a resolution of 4000×2500 pixels. Calculating vertical Δ:

$$\Delta = \frac{24\text{mm}}{2500} \approx 0.01\text{mm}$$

Thus, the resolving power is $R_p = \frac{1}{2 \times 0.01\text{mm}} = 50$ lines/mm.

If we capture an object at distance $u = 5000$mm with focal length $f = 50$mm, what is the minimum distinguishable object size? Rays from adjacent distinguishable lines have an angular separation θ. For small angles:

$$2\Delta \approx f\theta \implies \theta = \frac{2\Delta}{f} = \frac{0.02\text{mm}}{50\text{mm}} = 0.0004 \text{ radians}$$

The minimum object size is:

$$\text{object size} \approx u\theta = 5000\text{mm} \times 0.0004 = 2\text{mm}$$

3.2.5 Typical Lens Issues

- **Barrel distortion**: Caused by wide-angle lenses; straight lines curve outward from the center.
- **Chromatic aberration**: The same wavelengths are refracted differently, resulting in a missmatch of colors at the edges of objects.
- **Vignetting**: Light falloff causes the image corners to appear darker than the center.

3.3 Projection

By the time we capture an image, we have already projected the 3D world onto a 2D plane. Considering the time dimension, we formalize projection as:

$$f : \mathbb{R}^4 \to \mathbb{R}^3$$

$$(X, Y, Z, t) \mapsto (x, y, t)$$

We will discuss the two main projection models used in computer vision: **perspective projection** and **orthographic projection**.

3.3.1 Perspective Projection

Perspective projection models how objects appear smaller as distance increases. In this specific coordinate convention, the image plane is at the origin $(0, 0, 0)$ and the lens center is at $Z = f$.

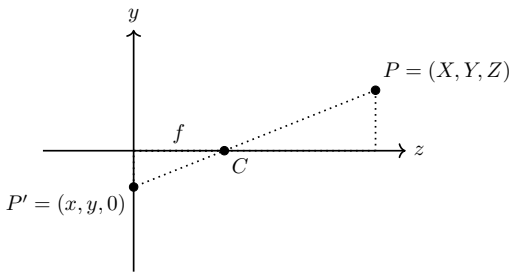


Figure 12: Perspective Projection

The similar triangles formed by points P, C, and P' give:

$$\begin{cases} Z' = 0 \\ \\ \dfrac{f}{Z-f} = \dfrac{x}{X} \\ \\ \dfrac{f}{Z-f} = \dfrac{y}{Y} \end{cases}$$

Therefore, the projection of $P = (X, Y, Z)$ is:

$$P' = \left(f\frac{X}{Z-f},\ f\frac{Y}{Z-f},\ 0 \right)$$

Because perspective projection maps infinitely many 3D points lying on the same viewing ray to the same image point, depth cannot be recovered from a single image alone without additional constraints or multiple views (e.g., stereo vision).

Simplified Perspective Projection

To avoid an inverted image, we can place a virtual image plane in front of the lens at distance f.

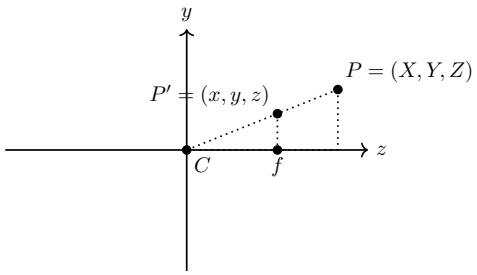


Figure 13: Simplified Perspective Projection.

The simplified equations map the 3D point $P = (X, Y, Z)$ to:

$$P' = \left(f\frac{X}{Z},\ f\frac{Y}{Z},\ 0 \right)$$

This simplification is highly practical and standard when $f \ll Z$.

3.3.2 Orthographic Projection

Orthographic projection assumes that light rays generated by the scene are parallel and perpendicular to the image plane. This allows for a simple projection where perspective distortion is ignored.

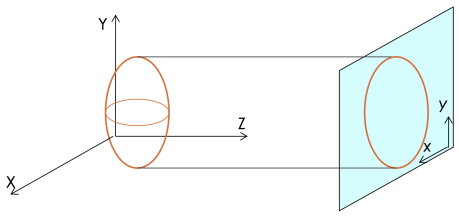


Figure 14: Orthographic Projection model.

This is valid when the object's depth is negligible compared to its distance from the camera (e.g., satellite imaging).

$$P' = (X, Y, 0)$$

In matrix form:

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

3.4 Illumination Model

In computer vision, lighting plays a crucial role in determining the appearance of objects in an image. Once geometry is established, illumination dictates the observed pixel intensities. Light hitting an object can be **absorbed**, **reflected**, or **transmitted**. Because standard cameras rely purely on reflected light, modeling surface reflection is a fundamental task.

3.4.1 Surface Reflection Types

Reflection behavior is governed by surface properties (e.g., roughness, texture, and material) and is generally categorized into two main types:

- **Specular**: Energy is reflected in a concentrated direction, typical of glossy or mirror-like surfaces. This causes highlights that depend heavily on the observer's viewpoint.
- **Diffuse**: Energy is scattered uniformly in all directions, characteristic of matte surfaces. The observed intensity is independent of the observer's position.

In computer vision, perfectly diffuse surfaces are generally easier to model and analyze. Glossy surfaces introduce specular highlights, which are view-dependent and can complicate tasks like feature extraction or 3D reconstruction.

3.4.2 The Lambertian Reflectance Model

To simplify illumination modeling, we often assume a **Lambertian surface** (a perfectly diffuse or matte surface) illuminated by a single, distant point light source. Because the light source is distant, the incoming rays are considered parallel and can be represented by a single directional vector s.

According to Lambert's Cosine Law, the reflected intensity i for a perfectly diffuse surface is proportional to the cosine of the angle α between the **surface normal** vector n and the **light direction** vector s. Since the object reflects light uniformly in all directions, specular components are completely ignored and the observed intensity is independent of the observer's position.

The model is expressed as:

$$i = \rho(s \cdot n) \tag{3.6}$$

Expanding the dot product, we get:

$$i = \rho \|n\| \|s\| \cos(\alpha) \tag{3.7}$$

where $\rho \in [0, 1]$ represents the surface *albedo* (the inherent fraction of incoming light that the material reflects). If n and s are treated as normalized unit vectors, the equation simplifies to $i = \rho \cos(\alpha)$.

Additionally, if $(s \cdot n) \leq 0$, the surface normal points away from the light source. This means the point is self-shadowed and receives no direct illumination, resulting in an observed intensity of zero.

4 Motion detection

In this section we will discuss the analysis of a scene, not only in terms of a static image, but also in terms of temporal evolution of it. **Motion** is one of the most important features of a scene, and we are usually interested in detecting:

- **Moving objects**, which are the objects that are moving in the scene;
- **Camera motion**, which is the motion of the camera itself.

Motion can be formally defined as the change of position of an object in a scene over time. In the 3D case, it can be represented as:

$$\boldsymbol{D}(\boldsymbol{X}, t_1, t_2) = \boldsymbol{X'} - \boldsymbol{X} = [\Delta x, \Delta y, \Delta z] \tag{4.1}$$

When projected into the 2D image plane we get:

$$\boldsymbol{d}(\boldsymbol{x}, t_1, t_2) = \boldsymbol{x'} - \boldsymbol{x} = [\Delta x, \Delta y] \tag{4.2}$$

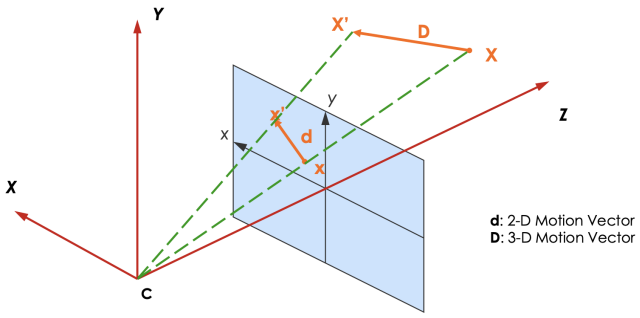


Figure 15: Projection of movement from 3D to 2D.

From this, if we consider a single point, we can derive that the displacement we perceive in the image plane can be quite different from the actual 3D displacement, due to the projection of the 3D scene onto the 2D image plane.

The final goal of the analysis of motion is to be able to extract the **motion field**, which is the set of all the displacements of all the points in the scene, understand if it really corresponds to the actual motion of the objects in the scene, and understand if it was caused by the motion of the camera itself or by the motion of the objects in the scene.

Figure 16: Example of motion detection.

An example In this example, we can clearly identify three main components of the motion field:

- The motion of the face, which moved from left to right between the two frames. In fact, in the **motion field**, we can see the arrows pointing from left, as signaling that the "point" of frame 2 is to the right of the corresponding "point" in frame 1.
- Static section, corresponding to the background, which is not moving, and therefore has no motion field. If arrows were present, they would represent the motion of the camera, as the background of this example is static;
- The motion of the head. Since the safety helmet is mostly flat, the algorithm is not able to correctly identify the motion of the head. In the motion field, this results in a "noisy" motion field, with arrows pointing in different directions, and not corresponding to the actual motion of the head.

Applications Motion detection is a fundamental task in computer vision, and it has many applications, such as:

- **Detection of moving objects**, which is useful for surveillance, traffic monitoring, and many other applications;
- **Segmentation of moving objects**;
- **Recognition**, we could filter different objects based on their motion (e.g., we could recognize a car from a pedestrian based on their motion);
- **Filtering**;
- **Compression**, in case of videos, the motion component can be used to compress the video.

In general, it is used to detect changes of the image intensity, in the scene over time.

4.1 Optical Flow

The motion detection we have described so far can be formalized trough the concept of **optical flow**.

First, we define the assumption taken in the optical flow:

- **Brightness constancy**: the object illumination does not change over time;
- **Small motion**: the motion between two frames is small enough;
- **Spatial coherence**: each point $[x, y]$ in the image plane is shifted to a new position $[x + \Delta x, y + \Delta y]$. This impose that the motion is only a translation, and that there is no rotation or scaling.

This is usually not true in real scenarios, but it is a good approximation for small motions.

4.1.1 Optical Flow Function

The optical flow function is defined as:

$$\psi(x + \Delta x, y + \Delta y, t + \Delta t) = \psi(x, y, t) \tag{4.3}$$

This means that the intensity of the point $[x, y]$ at time t is the same as the intensity of the point $[x + \Delta x, y + \Delta y]$ at time $t + \Delta t$, as long as the motion is small enough. This is the brightness constancy assumption.

The Taylor expansion states that for a function $f(x + \Delta x)$, we can approximate it as:

$$f(x + \Delta x) \approx f(x) + \frac{\partial f(x)}{\partial x} \Delta x$$

By applying the Taylor expansion to the left-hand side of the equation 4.3, we get:

$$\psi(x + \Delta x, y + \Delta y, t + \Delta t) \approx \psi(x, y, t) + \frac{\partial \psi}{\partial x} \Delta x + \frac{\partial \psi}{\partial y} \Delta y + \frac{\partial \psi}{\partial t} \Delta t$$

By substituting this into the equation 4.3, we get:

$$\frac{\partial \psi}{\partial x} \Delta x + \frac{\partial \psi}{\partial y} \Delta y + \frac{\partial \psi}{\partial t} \Delta t = 0$$

By dividing both sides by Δt, we get:

$$\frac{\partial \psi}{\partial x} \frac{\Delta x}{\Delta t} + \frac{\partial \psi}{\partial y} \frac{\Delta y}{\Delta t} + \frac{\partial \psi}{\partial t} = 0$$

By defining the velocity of the point as $v_x = \frac{\Delta x}{\Delta t}$ and $v_y = \frac{\Delta y}{\Delta t}$, we get the optical flow equation:

$$\frac{\partial \psi}{\partial x} v_x + \frac{\partial \psi}{\partial y} v_y + \frac{\partial \psi}{\partial t} = 0 \tag{4.4}$$

This implies that the variation of the intensity of a point in the image plane is related to the velocity of the point in the image plane.

If we decompose the velocity into its components (normal and tangential with respect to the surface of the object), we can see that only the normal component of the velocity can be detected, since the tangential component does not cause any change in the intensity of the point. If we think about this from a practical point of view, this means that we can only detect the motion of the borders of the objects, while the motion of the internal points of the objects cannot be detected as they would replace the same intensity values in the image plane, and therefore they would not cause any change in the intensity of the point. This is the aperture problem, a very well-known issue of the optical flow, which is detailed in the challenges below.

4.2 Problems and Challenges

The motion detection task is quite challenging, and the solution might be **not unique**.

In fact, the motion is based on the **velocity** of the points in the scene, over two frames. This might not reflect the actual motion of the objects in the scene, as it might also be caused by **camera motion** or by **illumination changes**.

4.2.1 Aperture

Not all motion can be detected by analyzing the changes in the image intensity. In fact, only displacements of the borders of the objects can be detected, while the motion of the internal points of the objects cannot be detected.

Also, borders can only detect the motion in the direction perpendicular to the border itself (in the direction of the intensity gradient), while for angles, we can detect motion in both directions.

4.2.2 Real vs Apparent movement

Also, being the motion perceived by changes in the scene, we could have two possible situations for which the motion field is not reflecting the actual motion of the objects in the scene:

- **Constant illumination**, in which a sphere is moving in the scene (rotating around its own axis), but the illumination is constant. In this case, the

motion field will be zero, as there are no changes in the image intensity, even though the sphere is moving.

- **Changing illumination**, in which the sphere is not moving, but the illumination source rotates around the sphere. In this case, the motion field will be non-zero, as there are changes in the image intensity, even though the sphere is not moving.

4.2.3 Occlusion

Another non-trivial problem is the occlusion. This problem occurs when an object in the background is partially or completely occluded by an object in the foreground. This can cause problems in the motion detection, as a static background might be perceived as moving, due to the occlusion of the foreground object.

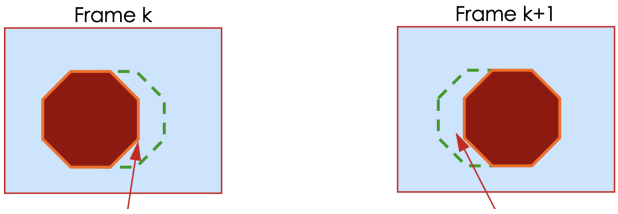


Figure 17: Example of occlusion.

4.3 Motion Detection in practice

There are two main approaches to motion detection in practice:

- **Intensity-based methods**, which are based on the analysis of the changes in the image intensity over time;
- **Feature-based methods**: based on the analysis of the changes in the features of the objects in the scene over time. This requires additional steps of feature extraction and matching, but it can be more robust to changes in the illumination and to occlusions.

4.3.1 Change Detection

The change detection is a simple approach to motion detection. It takes as input two frames of a video, taken at different time instants K and $K-1$, and it computes the difference between the two frames. This can be done by simply following the following steps:

- **Subtraction**: we compute the difference between the two frames, pixel by pixel. The resulting image will have high intensity values in the areas where there is a change in the intensity, and low intensity values in the areas where there is no change.
- **Thresholding**: we apply a threshold to the resulting image, to obtain a binary image. The pixels with intensity values above the threshold will be set to 1, and the pixels with intensity values below the threshold will be set to 0.

From a computational point of view, this approach is very simple and fast. We can formalize it as:

- **Frame difference**: corresponds to the subtraction of two images $I_K - I_{K-1}$. We can define the **frame differencing operation** as:

$$FD_{K,K-1}(x,y) = s(x,y,k) - s(x,y,k-1) \tag{4.5}$$

  If FD is not null, then a change has been detected in the pixel $[x,y]$;
- **Tresholding**: to filter out the noise, we can apply a threshold to the resulting image. We can define the **thresholding operation** as:

$$T(x,y) = \begin{cases} 1 & \text{if } FD_{K,K-1}(x,y) > \tau \\ 0 & \text{otherwise} \end{cases} \tag{4.6}$$

This algorithm allows to quickly adapts to changes in the backgrounds and to the stopping of the moving objects. It can detect the contours of the moving objects, but it is not able to detect the motion of the internal points of the objects, due to the aperture problem.

To resolve these problems, usually the time scale of the analysis is increased, by considering more distant frames. This allows to detect the motion of the internal points, but for fast moving objects, it can cause problems.

4.3.2 Background subtraction

The background subtraction is a more sophisticated approach to motion detection. It is based on the idea of modeling the background of the scene as a static image (acquired a priori).

The algorithm can be formalized as follows:

Algorithm 1 Background subtraction algorithm

1: $B = I(0)$; ▷ Initialize background model
2: **for** each frame $I(t)$ **do**
3: $D = |I(t) - B|$; ▷ Difference between current frame and background
4: $M = Threshold(D)$; ▷ Apply thresholding to obtain the motion mask
5: **end for**

This allows for a better representation of the moving objects in the scene, as it can filter out the static background, but it has a few limitations, such as:

- **Modelling the background**: the background model might not be accurate, especially in case of dynamic backgrounds;
- **Changes in the background**: if a background object starts moving, both the real object and its "ghost" (since the background model is not updated) will be detected as moving objects;
- **Illumination changes**: if the illumination changes, the background model is not able to adapt to it as it is based on a static image;
- **Camera motion**: if the camera is moving, again the background model is not able to adapt to it.

4.3.3 Adaptive Background Subtraction

To overcome the limitations of the background subtraction, we can use an adaptive background subtraction approach. In this approach, the background model is updated over time, to adapt to changes in the background and to changes in the illumination. The update of the background model can be formalized as:

$$B_t = \alpha I(t) + (1 - \alpha) B_{t-1} \tag{4.7}$$

Where α is a parameter that controls the update rate of the background model:

- If α is close to 1, the background model will be updated quickly, going back to the **change detection** approach;
- If α is close to 0, the background model will be updated slowly, maintaining a more stable representation of the background.

4.3.4 Gaussian Average

Instead of relying on a single background model, we can use a Gaussian Model to represent each pixel in the background. In this approach, we model each pixel as a Gaussian distribution, and we update the parameters (μ, σ^2) of the distribution over time, with the same approach of the adaptive background subtraction.

We can formalize the Gaussian model as follows:

$$\mu_t = \alpha I_t + (1 - \alpha) \mu_{t-1} \tag{4.8}$$

This model can adapt to changes in the background and to changes in the illumination, by "absorbing" the changes in the background into the parameters

of the Gaussian distribution. The speed of adaptation is controlled by the parameter α, as in the adaptive background subtraction approach.

This model allows recognizing the moving objects in the scene, even in case of dynamic backgrounds. We can instead say that a pixel belongs to the foreground if the intensity value of the pixel is far from the mean of the Gaussian distribution, by a certain threshold:

$$|I_t - \mu_t| > k\sigma_t \tag{4.9}$$

Where k is the threshold parameter, and σ_t is the standard deviation of the Gaussian distribution at time t.

Even though the model is constantly updated, usually an initialization phase is required, in which the background model is learned from a sequence of frames. This allows to have a more accurate representation of the background, and to reduce the number of false positives in the motion detection.

Improving the Gaussian Average From the equation 4.8 of the Gaussian average, we can see that the update of the mean is based on both background and foreground pixels, with no distinction between them.

To avoid foreground pixels to affect the update of the background model, we can use a **selective update** approach, in which only the pixels that are classified as background are used to update the background model. This can be formalized as:

$$\mu_t = M\mu_{t-1} + (1 - M)(\alpha I_t + (1 - \alpha)\mu_{t-1}) \tag{4.10}$$

Where M is a binary mask that indicates which pixels are classified as background (1) and which pixels are classified as foreground (0). If the pixel is classified as foreground, the background model is not updated at all, while if the pixel is classified as background, the background model is updated as in the Gaussian average approach.

4.3.5 Mixture of Gaussian

There are cases in which a single Gaussian distribution is not enough to model the background. For example, in case of rain, snow, or waving trees, the background can have multiple modes, and a single Gaussian distribution is not able to capture all the variations in the background. These are known as multimodal backgrounds, and they require a more complex model to be able to capture all the variations in the background.

In this case, we can use a mixture of Gaussian distributions to model the background. In particular, we can model the background as a mixture of K Gaussian

distributions, where each distribution represents a different mode of the background. More formally, we can define the mixture of Gaussian model as:

$$P(X_t) = \sum_{i=1}^{K} w_{i,t} \mathcal{N}(X_t | \mu_i, \Sigma_i) \tag{4.11}$$

Where:

- $P(X_t)$ is the probability of the pixel intensity X_t at time t;
- $w_{i,t}$ is the weight of the i-th Gaussian distribution at time t, which represents the importance of the i-th mode of the background at time t;
- $\mathcal{N}(X_t | \mu_i, \Sigma_i)$ is the Gaussian distribution with mean μ_i and covariance Σ_i.

We can make two assumptions to simplify the model:

- **Independence of the color channels**: we can assume that each color channel is independent from the others. Therefore, the covariance matrix Σ_i can be simplified to a diagonal matrix, which reduces the number of parameters to be estimated.
- **Equal variances**: we can also assume that all the channels have the same variance. This way, we can replace the covariance matrix Σ_i from the model, in favor of a single variance σ_i^2.

Identify each Distribution Once we have defined the K Gaussian distributions, we need to rank them based on their weights $w_{i,t}$, amplitude and standard deviation, to determine which distributions represent the background and which distributions represent the foreground. Usually, the distributions with the highest weights and lowest standard deviation are considered as background distributions, while the distributions with the lowest weights and highest standard deviation are considered as foreground distributions. Formally, the first B distributions that satisfy the following condition are considered as background distributions:

$$\sum_{i=1}^{B} w_{i,t} > T \tag{4.12}$$

Where T is a threshold parameter that controls the sensitivity of the model to changes in the background.

Point classification Given the mixture of Gaussian model, we can check each pixel intensity X_t against the distributions:

- If X_t matches one of distributions, it is classified accodingly. X_t is said to match a distribution if the distance between X_t and the mean of the distribution is less than a certain threshold:

$$|X_t - \mu_i| < k\sigma_i$$

- If X_t does not match any of the distributions, the distribution with the lowest weight is replaced with a new distribution with mean equal to X_t, high standard deviation, and low weight.

Update Model parameters Also in this case, the parameters of the model are updated over time, to adapt to changes in the background and to changes in the illumination. In particular, we are interested in updating the weights of the distributions, since the means and the standard deviations are updated just as in the Gaussian average approach, but only for the distribution that matches the pixel intensity X_t.

The update of the weights can be formalized as:

$$w_{i,t} = (1 - \alpha)w_{i,t-1} + \alpha M \tag{4.13}$$

Where M is a binary variable that indicates whether the distribution matches the pixel intensity X_t (1 if it matches, 0 otherwise). This update allows to increase the weight of the distribution that matches the pixel intensity X_t, and to decrease the weight of the distributions that do not match the pixel intensity X_t.

5 Motion Tracking

While motion detection identifies the presence of movement, **motion tracking** aims to follow specific objects across consecutive frames. This process involves establishing temporal correspondence by associating detected entities in the current frame with their instances in subsequent frames, allowing us to reconstruct the trajectory of each object over time.

Motion tracking is essential across various domains, including:

- **People monitoring:** analyzing crowd dynamics or tracking a specific individual;
- **Vehicle tracking:** monitoring the movement of vehicles in traffic surveillance;
- **Biological applications:** tracking the movement of animals or cells in scientific research;

The primary objective is often **object tracking**, which maintains the persistent identity of a target over time. Depending on the application, tracking can be performed in different dimensionality:

- Tracking 2D coordinates in the image plane (e.g., using bounding boxes or centroids);
- Tracking 3D coordinates in real-world space (requires multiple cameras or depth sensors);
- Determining the pose or articulation of complex objects, such as human skeletal tracking.

5.1 Foundations: Detection, Association, and Challenges

Before diving into specific tracking methodologies, it is crucial to understand how raw pixel changes are converted into trackable objects, how these objects are linked across frames, and the common challenges that arise during this process.

5.1.1 Blob Extraction and Post-Processing

Following motion detection, the resulting binary masks often contain "noise", in the form of fragmented regions, isolated pixels, or holes within moving objects caused by imperfect segmentation. To transform these raw pixels into actionable data, we perform **blob extraction** (also known as Connected Component Labeling).

This process aggregates neighboring foreground pixels into coherent, labeled regions called **blobs**, which represent the physical objects in the scene. The final goal is that each blob corresponds to a single moving object, providing a high-level representation that can be easily tracked over time.

To ensure these blobs accurately reflect the geometry of the targets, the extraction process typically involves the following refinement steps:

- **Morphological Filtering:** Applying *erosion* to remove isolated noise (small pixel clusters) and *dilation* to bridge gaps and fill internal holes within an object.
- **Connectivity Analysis:** Grouping pixels based on 4-way or 8-way adjacency to assign a unique ID to each spatially distinct moving entity.
- **Feature Extraction:** Once a blob is defined, we calculate its geometric properties, such as the **centroid**, **bounding box**, **area**, and **aspect ratio**.

By filtering out blobs that are too small to be objects of interest, we provide the downstream tracking algorithms with a clean, high-level representation of the scene.

5.1.2 Target Association

Once we have the blobs representing the detected objects, we need to associate them across consecutive frames to maintain the identity of each object over time. This process is known as **target association** and is actually common to all tracking methods.

In general, detection is not carried out on a frame-by-frame basis, but rather on a **temporal window** of frames, since it can be demanding in terms of computational resources. This means that the tracking algorithm must be able to handle cases where an object is not detected in every single frame, which adds complexity to the association process.

Correspondence is typically achieved through **data association algorithms** that match the detected blobs based on their spatial proximity and appearance features. Such methods rely on the **optical flow** assumption, which states that the motion of objects between frames is small enough to allow for reliable matching based on their positions and appearances.

It's important to keep in mind that cameras and frame rates must be chosen to satisfy this assumption, as large displacements can lead to tracking failures.

Different approaches to target association include:

- **Nearest Neighbor:** Assigning each detected blob to the closest track based on spatial distance between centroids;

- **Overlapping:** Matching blobs based on the degree of overlap between their bounding boxes. This has issues with scale changes;
- **Bounding Box - Overlap:** Similar to the previous method, but consider the bounding box of the track instead of the blob.

There are no best practices for target association, and the choice of method depends on the specific application and the characteristics of the scene being analyzed.

5.1.3 Splitting and Merging

Very similar but distinct problems that can arise during tracking are **splitting** and **merging**.

Splitting When multiple blobs are present in the same frame, we might encounter one of two problematic scenarios:

1. **Single object - multiple blobs:** this occurs when an object is fragmented into several blobs due to imperfect segmentation.
2. **Multiple objects - single blob:** this happens when two or more objects are merged into a single blob, often due to occlusion or proximity.

These 2 situations are different faces of the same problem, which is called **splitting**.

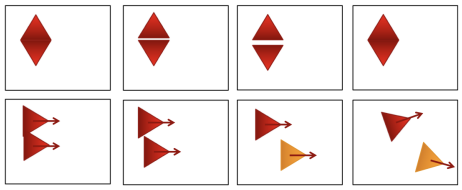


Figure 18: Example of splitting: a single object is fragmented into multiple blobs.

Before understanding which of the two cases we are facing, evidence need to be collected over a temporal window of frames. This way, we can analyze the behavior of the blobs over time to determine whether they represent a single object or multiple objects.

Merging On the other hand, **merging** occurs when multiple objects are detected as a single blob, due to proximity or occlusion. An example of this is when a person which is entering a scene. Initially, feet and arms might be detected as separate blobs. As the person moves further into the scene, these blobs might merge into a single blob representing the whole body.

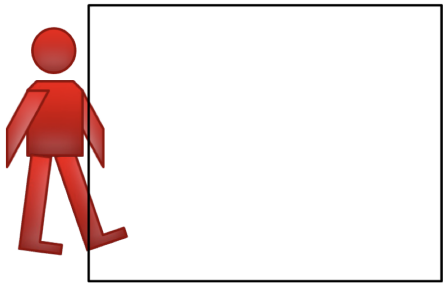


Figure 19: Example of merging: multiple objects are merged into a single blob.

In both splitting and merging, it is required to analyze the temporal evolution of the blobs to determine whether they represent a single object or multiple objects, and to adjust the tracking accordingly. Usually, matching features such as color, shape, or motion patterns can help to resolve ambiguities in these cases.

5.1.4 Occlusion

Moving object can be partially or fully occluded by each other when they are in close proximity. This can lead to tracking failures, as one object disappears from the scene, and bigger blobs may appear, which might have no resemblance to the original objects.

This is something that need to be handled before the occlusion occurs, by analyzing the proximity of the objects and predicting potential occlusions. This way, the system can prepare for the occlusion by maintaining the identity of the involved objects and anticipating their reappearance after the occlusion.

Usually, the system suspends the tracking of the involved objects during the occlusion, and then re-identifies them once they reappear as separate entities in the scene. No model update should be performed during the occlusion. Re-identification is usually done by matching features with the pre-occlusion appearance of the objects.

5.2 2D Tracking Methodologies

In scenes with a single moving object, tracking can often be achieved by simply following the object's centroid. However, multi-object scenarios require sophisti-

cated association algorithms to prevent identity switches. Common 2D tracking approaches include:

- **Region-based:** Associating areas based on global features like color or texture.
- **Contour-based:** Tracking the object's based on the shape of its boundary.
- **Feature-based:** Tracking distinct interest points, such as Harris corners or SIFT features.
- **Template-based:** Matching a predefined visual patch of the target across frames.

5.3 Region-Based Tracking

Region-based tracking follows image patches with a **uniform appearance**. This method is computationally efficient and well-suited for real-time systems, providing a robust balance between speed and discriminative power.

The standard implementation utilizes a **color histogram** as a visual signature. The workflow typically involves:

1. Segmenting the region of interest (ROI) from the background.
2. Computing the color distribution (histogram) of the ROI.
3. Searching for the most similar histogram in the subsequent frame to locate the object.

While effective for targets with distinctive colors (each object needs to have a unique color distribution), this approach is sensitive to illumination changes. To mitigate this, practitioners often employ the **HSV color space** (which separates luminance from chromaticity) or implement **dynamic model updates** to adapt the histogram over time.

5.3.1 Part-Based Models

A single global histogram may fail when tracking complex, non-rigid objects like humans. **Part-based models** decompose the object into several sub-regions (e.g., head, torso, and legs), each with its own local histogram.

This decomposition provides two major advantages:

- **Robustness to Occlusion:** If the legs are hidden by an obstacle, the system can maintain the track using the head and torso signatures.

- **Spatial Verification:** It prevents confusion between objects with similar colors but different configurations. Take in consideration the case of two people, one wearing a white shirt and black pants, and the other wearing a black shirt and white pants. A global histogram might struggle to differentiate them, but a part-based model can leverage the spatial arrangement of colors to maintain accurate tracking.

5.3.2 Implementation and Similarity Metrics

The tracking process evaluates the similarity between the **tracked model** (O^t) in the current frame and a stored **reference model** (O^r), which is updated over time to adapt to changes in the object's appearance.

Common metrics for histogram comparison include:

- **Histogram Intersection:** Measures the correlation between two distributions.

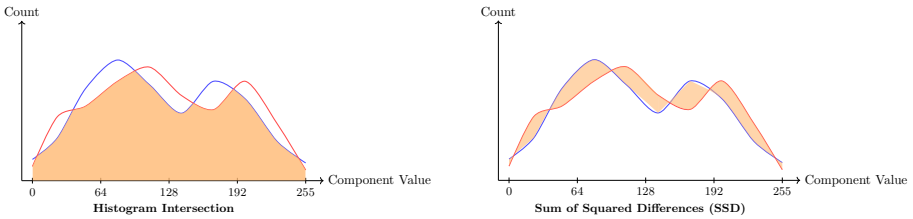
$$\cap(O^t, O^r) = \sum_{n=1}^{U} \min(O_n^t, O_n^r)$$

  This simply returns the area of the intersection of the two histograms, which is a measure of their similarity.

- **Sum of Squared Differences (SSD):** Measures the Euclidean distance between histogram bins.

$$SSD(O^t, O^r) = \sum_{n=1}^{U} (O_n^t - O_n^r)^2$$

  This instead returns the area of the difference between the two histograms, which is a measure of their dissimilarity.



To improve robustness and reduce computational cost, bins typically represent color ranges rather than individual values. This is done by grouping pixel values into discrete intervals (e.g., 16 or 32 bins per channel), which allows for a more compact representation of the color distribution while still capturing essential information for tracking.

5.3.3 Addressing Environmental Noise: Shadows

Shadows represent a significant challenge in region-based tracking because they alter the pixel values of the background, often leading to false positives in motion detection.

To prevent a shadow from being tracked as a separate entity or distorting the object's histogram, we assume that a shadow primarily reduces **brightness** while leaving **chromaticity** (Hue and Saturation) relatively intact. By performing tracking in the HSV space and ignoring the Value (V) component, the system can become significantly more shadow-invariant, improving the robustness of the tracking process in outdoor environments or scenes with variable lighting conditions.

5.4 Feature Based Tracking

The idea behind feature-based tracking is to replace the information carried by the histogram of the whole object with a set of distinctive local **features**, extracted from the object itself. We should then be able to track these features over time, and use their movement to infer the movement of the whole object. These features are typically points of interest, such as corners, edges, or blobs, that can be reliably detected and tracked across frames.

Considering a set of images $I = \{I_1, I_2, ..., I_n\}$, we can extract a set of features F_i from each image I_i. The position of each feature in a given image can be represented as a vector of coordinates $f_i(x_i, y_i)$. The objective of feature-based tracking is to determine the displacement vector $d_i = (dx_i, dy_i)$ that best estimates the movement of feature f_i from one frame to the next, such that:

$$f_{i+1}(x_i + dx_i, y_i + dy_i)$$

5.4.1 Good Features to Track

Not all features are equally suitable for tracking. The **Good Features to Track** algorithm identifies features that are likely to be stable and distinctive across frames, making them ideal candidates for tracking. This method is based on the concept of **corner detection**, which identifies points in the image where the intensity changes significantly in multiple directions.

For each pixel in the image $p(x, y)$ we compute the **structure tensor** Z_p:

$$Z_p = \begin{bmatrix} \sum_W J_x^2 & \sum_W J_x J_y \\ \sum_W J_x J_y & \sum_W J_y^2 \end{bmatrix}$$

where J_x and J_y are the gradients of the image in the x and y directions, respectively, and the summation is performed over a window W centered around the pixel p.

This is basically the Jacobian Matrix of the image gradients, which captures the local intensity variation around the pixel. The eigenvalues of the matrix, λ_1 and λ_2, provide a measure of the local intensity variation around the pixel. We can identify three cases based on the values of the eigenvalues:

- **Flat region:** if both eigenvalues are small, the pixel is in a flat region with little intensity variation, making it unsuitable for tracking;
- **Edge:** if one eigenvalue is large and the other is small, the pixel is on an edge, which can be tracked but may not be stable due to the lack of variation in one direction (Aperture Problem);
- **Corner:** if both eigenvalues are large, the pixel is in a corner, which is ideal for tracking because it has significant intensity variation in multiple directions.

A pixel is considered a good feature to track if the smallest eigenvalue $\lambda_{min} = \min(\lambda_1, \lambda_2)$ is above a certain threshold, indicating that changes are in multiple directions, which is characteristic of corners.

5.4.2 How to track features

Once we have identified the good features to track, we need to track their movement across frames. Ideally, we would expect that the position of a feature in the next frame can be predicted by adding a displacement vector to its current position:

$$I_i(Df - d) = I_{i+1}(f)$$

where I_i, I_{i+1} are successive frames, f is the position of the feature in the current frame, D is the deformation matrix and d is the displacement vector we want to estimate.

However, due to the noise, the equality cannot hold perfectly. Assuming that the motion between frames is small, we can find an approximation of the movement of the feature, instead of trying to find the exact displacement vector d. This approach simplifies the problem to finding the displacement vector d that minimizes the difference between the two images:

$$\epsilon = \int \int_W [I_i(f - d) - I_{i+1}(f)]^2 \omega(f)\, df$$

where $\omega(f)$ is a weighting function that gives more importance to the pixels closer to the feature point, and less importance to the pixels farther away.

The value of d that minimizes this error can be found by simply taking the derivative of ϵ with respect to d and setting it to zero.

5.5 Lucas-Kanade Optical Flow

The Lucas-Kanade method is an improvement over the basic feature tracking approach. It uses a two-frame differential method to estimate the optical flow, which is the apparent motion of brightness patterns in the image.

Consider $u = (u_x, u_y)$ in frame I_i and $v = (v_x, v_y)$ in frame I_{i+1}. The goal is to find the displacement vector d such that:

$$v = u + d$$

Where d is the vector that minimizes:

$$\epsilon(d) = \epsilon(d_x, d_y) = \sum_{x=u_x-\gamma_x}^{u_x+\gamma_x} \sum_{y=u_y-\gamma_y}^{u_y+\gamma_y} [I_i(x, y) - I_{i+1}(x + d_x, y + d_y)]^2$$

Where γ is the size of the window around the feature point that we are considering for tracking. This is simply the sum of squared differences between the pixel intensities in the two frames, within a window centered around the feature point.

The strong point of the Lucas-Kanade method comes from its pyramidal implementation, which allows it to handle larger displacements by creating a multiscale representation of the image.

5.5.1 Pyramidal implementation

The two key components that any feature tracker must consider are **accuracy** and **robustness**.

- **Accuracy:** relates to the ability to recreate as perfectly as possible the motion of the feature. Usually, a smaller window size γ is preferable to increase accuracy, since it focuses on a smaller area around the feature point.
- **Robustness:** relates to the sensitivity of the tracking algorithm to big displacements. Usually, a larger window size γ is preferable to increase robustness.

We can notice that the two objectives are in conflict with each other, since increasing the window size to improve robustness will decrease accuracy in detecting small movements, and vice versa.

The pyramidal implementation of the Lucas-Kanade method addresses this trade-off by creating a multiscale representation of the image, where each level of the pyramid corresponds to a different resolution.

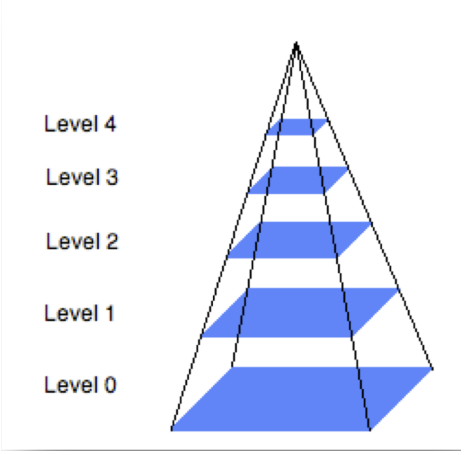


Figure 20: Pyramidal implementation of the Lucas-Kanade method.

We have that:

- At level 0, we have the original image with the highest resolution, which allows for accurate tracking of small movements;
- At higher levels, we have downsampled versions of the image. Due to the reduced resolution, bigger movements are now represented as smaller pixel displacements, allowing for more robust tracking of larger movements.

The L-th level of the pyramid is defined as a linear combination of the elements of the previous level, with more weight given to the central pixels and less weight given to the peripheral pixels.

$$\begin{aligned} I^L(x,y) = &\frac{1}{4} I^{L-1}(2x,2y) + \\ &\frac{1}{8}\Big(I^{L-1}(2x-1,2y) + I^{L-1}(2x+1,2y) + I^{L-1}(2x,2y-1) + I^{L-1}(2x,2y+1)\Big) + \\ &\frac{1}{16}\Big(I^{L-1}(2x-1,2y-1) + I^{L-1}(2x+1,2y-1) + I^{L-1}(2x-1,2y+1) + I^{L-1}(2x+1,2y+1)\Big) \end{aligned}$$

At each level of the pyramid, an initial guess g is obtained using the Lucas-Kanade method. The guess is then refined at each level and used to pre-translate the image patches for the next level, which allows for more accurate tracking of larger displacements.

The estimated displacement is then refined by propagating it down the pyramid:

$$g^{L-1} = 2(g^L + d^L)$$

Where:

- g^L is the displacement estimated at level L. It is multiplied by 2 to account for the fact that the resolution at level L is half of the resolution at level $L-1$, so the displacement needs to be scaled accordingly.
- d^L is the correction term computed at level L to fine-tune the estimate.

5.6 Bayesian Tracking

All the approaches seen so far are based on the assumption that the motion of the objects can be predicted by simply applying a deterministic model to the previous state of the system.

Bayesian tracking is a probabilistic approach to motion tracking that models the uncertainty in the tracking process. It tries to estimate the next position of the object (also known as the state of the system) over discrete time steps, given a sequence of observations.

State The state of the system at time t is represented by a random variable X_t, which includes information about the position, velocity, and acceleration along each axis of the tracked object, creating a 6-dimensional state space.

Its evolution can be modeled as a function of the previous state and a process noise term:

$$X_t = f_t(X_{t-1}, w_{t-1}) \tag{5.1}$$

Where:

- f_t is the non-linear state transition function that models how the state evolves over time;
- w_{t-1} is the process noise, representing the uncertainty in the state transition;
- X_{t-1} is the state of the system at the previous time step.

Take as example the case of a person walking in a scene. If no noise is involved, the state transition function f_t would simply predict the next position of the person based on their current velocity and direction of movement. Instead, we need to account for the fact that the person's movement might not be perfectly predictable, and that there might be external factors that can affect their trajectory (process noise).

Observation The observations at time t are represented by another random variable Z_t, which includes the measurements obtained from the sensors. It can be represented as a function of the current state and an observation noise term:

$$Z_t = h_t(X_t, v_t) \tag{5.2}$$

Where:

- h_t is the non-linear observation function that models how the state is transformed into observations;
- v_t is the observation noise, generated by the sensors and processing algorithms;
- X_t is the state of the system at the current time step.

5.6.1 How it works

The Bayesian tracking algorithm is a recursive filter that updates the system's state estimate whenever a new observation Z_t is recorded. It refines the predicted state X_t by correcting it with the new measurement.

The process begins with an initial probability distribution, or **prior**, $p(X_0)$, representing the baseline knowledge of the system.

The objective is to compute the **posterior distribution** $p(X_t|Z_t)$, which represents the updated belief of the state at time t given all the measurements up to that point.

The estimation cycle consists of two main steps:

1. **Prediction:** predict the next state based on the previous state and the state transition model, resulting in a prior distribution (not based on the observation) for the current time step.
2. **Update:** refine the predicted state using the new observation and the observation model, resulting in the posterior distribution (based on the observation) for the current time step.

Thanks to the recursive nature of the algorithm, the posterior distribution at time $t-1$ improves the prediction for time t, which in turn improves the posterior distribution at time t, and so on.

Prediction Step Using the state transition function from 5.1, the evolution of the system can be described by the conditional probability distribution:

$$p(X_t|X_{t-1})$$

which models the likelihood of transitioning from state X_{t-1} to state X_t.

Using the observation function from 5.2, the likelihood of observing the measurement Z_t, given the current state X_t can be expressed as:

$$p(Z_t|X_t)$$

To predict the state at time t, the algorithm propagates the posterior probability, obtained from the previous time step $p(X_{t-1}|Z_{1:t-1})$ forward. This results in the **prior distribution** for the current time step, which is computed by marginalizing over all possible previous states:

$$p(X_t|Z_{t-1}) = \int p(X_t|X_{t-1})p(X_{t-1}|Z_{t-1})dX_{t-1}$$

Update Step When the new observation Z_t becomes available, the likelihood $p(Z_t|X_t)$ is derived from the observation model in 5.2.

The prediction is then updated using Bayes' rule to obtain the new posterior:

$$p(X_t|Z_t) = \frac{p(Z_t|X_t)p(X_t|Z_{t-1})}{p(Z_t|Z_{t-1})}$$

where the denominator $p(Z_t|Z_{t-1})$ acts as a normalizing constant.

5.7 Kalman Filter

The Kalman Filter is a specific implementation of the Bayesian tracking framework, designed for linear systems with **Gaussian noise**.

In line to what we have seen in the bayesian tracking section, the Kalman Filter still consists of:

- A measurement of the state of the system (observation, subject to noise);
- A prediction of the next state (prior distribution, also subject to noise);
- An update of the state estimate based on new observations (posterior distribution).

It heavily relies on the assumption that both the process noise and the observation noise are Gaussian, which allows for a closed-form solution to the estimation problem.

The algorithm starts from a first measurement Z_0, with its associated noise variance $\sigma^2_{z_0}$. It assumes that the first prediction is equal to the first measurement, and that the first prediction error is equal to the noise variance of the first measurement:

$$\hat{X}_0 = Z_0$$
$$\sigma_0 = \sigma^2_{z_0}$$

The Kalman Filter then iteratively performs the prediction and update steps as new measurements become available, propagating the state estimate and its associated uncertainty over time. Given a new measurement Z_t, $\sigma^2_{z_t}$ at time t, the Kalman Filter performs the following steps:

1. **Prediction:** we add a displacement to the previous state estimate
$$\hat{X}_t = \hat{X}_{t-1} + K_t(Z_t - \hat{X}_{t-1})$$
where K_t is the **Kalman Gain**, which weights the influence of the new measurement relative to the current state estimate. It is computed as:
$$K_t = \frac{\sigma^2_{z_{t-1}}}{\sigma^2_{z_{t-1}} + \sigma^2_{z_t}}$$
We can see the prediction as a weighted average between the previous state estimate $\hat{X}_{t-1}$ and the new measurement Z_t, where the weights are determined by the relative uncertainties of the two sources of information.
2. **Update:** we update the error variance of the state estimate:
$$\frac{1}{\sigma^2_t} = \frac{1}{\sigma^2_{z_{t-1}}} + \frac{1}{\sigma^2_{z_t}}$$

The Kalman Filter provides a computationally efficient solution to the least-squares estimation problem. In particular, we are trying to find the Kalman Gain K_t that minimizes the mean squared error of the state estimate $\hat{X}_t$.

5.7.1 Kalman Filter in Practice

Now that we have seen the mathematical formulation of the Kalman Filter, let's discuss how it can be applied in practice for motion tracking.

State Model The state model can be represented as a linear function of the previous state and the control input, plus some process noise:
$$x_t = A_t x_{t-1} + B_t u_t + w_{t-1}$$
Where:

- x_t is the state vector at time t;
- A_t is the state transition matrix that models how the state evolves from time $t-1$ to time t;
- B_t is the control input matrix that models how external inputs u_t affect the state. For example, in the case of a vehicle tracking scenario, u_t could represent the acceleration or steering commands that influence the vehicle's motion;
- w_{t-1} is the process noise, assumed to be Gaussian with zero mean and covariance Q_t;

Usually, if we have no control input, we can simply ignore the term $B_t u_t$ in the state model, which simplifies it to:
$$x_t = A_t x_{t-1} + w_{t-1}$$

Observation Model The observation model can be represented as a linear function of the state, plus some observation noise:

$$z_t = H_t x_t + v_t$$

Where:

- z_t is the observation vector at time t;
- H_t is the observation matrix that models how the state is transformed into observations;
- v_t is the observation noise, assumed to be Gaussian with zero mean and covariance R_t;

5.7.2 The Recursive Stages

The filter iterates through the following equations to propagate the state estimate $\hat{x}_t$ and the error covariance matrix P_t:

1. Prediction Project the state and uncertainty forward, computing the prior state estimate $\hat{x}_t^-$ and its associated error covariance P_t^-:

$$\hat{x}_t^- = A_t \hat{x}_{t-1} + B_t u_t$$
$$P_t^- = A_t P_{t-1} A_t^T + Q_t$$

2. Update Once the new observation z_t is available, we compute the Kalman Gain K_t, update the state estimate $\hat{x}_t$, and refine the error covariance P_t:

- **Kalman Gain:** $K_t = P_t^- H_t^T (H_t P_t^- H_t^T + R_t)^{-1}$
- **State Update:** $\hat{x}_t = \hat{x}_t^- + K_t(z_t - H_t \hat{x}_t^-)$
- **Covariance Update:** $P_t = (I - K_t H_t) P_t^-$

5.7.3 Extended Kalman Filter

In the case of non-linear systems, the standard Kalman Filter is not applicable, since it relies on linearity assumptions. In object tracking scenarios, we often encounter non-linear dynamics and measurement models, which necessitate the use of the **Extended Kalman Filter** (EKF).

The EKF linearizes the non-linear functions around the current state estimate, allowing us to apply the Kalman Filter framework to non-linear problems. This is done by computing the **Jacobian matrices** of the state transition and observation functions, which serve as linear approximations of the non-linear models.

5.8 Particle Filter

Even though the EKF can handle non-linear systems, it still relies on the assumption of the optical flow and of Gaussian noise, which may not always hold in real-world tracking scenarios. The **Particle Filter** is a more general approach that can handle non-linear and non-Gaussian systems.

This is done by representing the set of possible alternative states of the system as a collection of samples, called **particles**, rather than a single state estimate with an associated covariance. Each particle represents a possible state of the system, and is associated with a weight that reflects the likelihood of that state given the observations.

The higher the number of particles, the closer our approximation will be to the true posterior distribution, but this also increases the computational cost of the algorithm.

5.8.1 Theoretical Definition

The Particle Filter defines a set of N particles $\{x_t^{(i)}, w_t^{(i)}\}_{i=1}^N$, where $x_t^{(i)}$ is the state of the i-th particle at time t, and $w_t^{(i)}$ is its associated weight.

These particles are selected from a proposal distribution $\pi(x_t|x_{t-1})$, which is chosen as an hyperparameter of the algorithm.

The a posteriori distribution of the state given the observations can be approximated as:

$$p(x_t|z_t) \approx \sum_{i=1}^{N} w_{t-1}^{(i)} \delta(x_t - x_{t-1}^{(i)})$$

where $\delta(x_t - x_{t-1}^{(i)})$ represents the distance between the state x_t and the state of the i-th particle $x_{t-1}^{(i)}$.

To update the particles and their weights, we perform the following steps:

1. **Sampling:** For each particle, we sample a new state from the proposal distribution;
2. **Weighting:** We update the weight of each particle based on the likelihood of the observation given the particle's predicted state:
$$w_t^{(i)} = w_{t-1}^{(i)} p(z_t|x_t^{(i)})$$
3. **Normalization:** the weights are normalized to ensure that they sum to 1. This is done by dividing each weight by the sum of all weights:
$$w_t^{(i)} = \frac{w_t^{(i)}}{\sum_{j=1}^{N} w_t^{(j)}}$$

4. **Resampling:** To prevent the problem of particle degeneracy, where a few particles have significant weights while others have negligible weights, we perform resampling. This involves drawing a new set of particles from the current set, with replacement, according to their weights. Each new particle is assigned a weight of $\frac{1}{N}$, and the process is repeated for the next time step.

This way, at each iteration, more particles are concentrated in regions of the state space that are more likely given the observations, allowing for a more accurate approximation of the posterior distribution.

For example, if we are tracking a person in a scene, and we have a new observation that indicates the person's direction of movement, the particles that are aligned with that direction will receive higher weights, while particles that are not aligned will receive lower weights and are less likely to be resampled in the next iteration.

6 Camera Geometry

We have already seen that the camera is a projection device. The location of the pixel in the image plane has a direct relationship with the location of the point in the 3D world. When we approximate the camera with a pinhole model, we might lose some information about the scene, such as the depth of the objects.

The position of the acquisition device plays a crucial role in the way we perceive the world. In this section we will try to give some explanation of these phenomena and try to give an explanation of how the camera geometry works.

6.1 2D Case

Typically, a pixel is represented as a 2D point in the image plane using its coordinates.

$$p = \begin{bmatrix} x, y \end{bmatrix}^T = \begin{bmatrix} x \\ y \end{bmatrix}$$

Often, we use homogeneous coordinates to represent points in the image plane. This allows us to represent points at infinity and perform projective transformations more easily. In homogeneous coordinates, a point is represented as a three-dimensional vector:

$$p = \begin{bmatrix} sx, sy, s \end{bmatrix}^T$$

where s is a scaling factor, commonly set to 1.

6.1.1 Affine Transformation

An affine transformation is a spatial transformation, represented as a multiplication of the **homogeneous point** with some predefined matrix. We can define different types of transformations, such as **scaling**, **rotation**, **translation** and a **composition** of them.

Scaling Scaling is a transformation that changes the size of an object, by multiplying the coordinates of each point by a scaling factor c. It is applied to all points in the image either uniformly or non-uniformly. The scaling transformation matrix S_{c_x,c_y} is defined as:

$$S_{c_x,c_y} = \begin{bmatrix} c_x & 0 \\ 0 & c_y \end{bmatrix}$$

where c_x and c_y are the scaling factors for the x and y directions, respectively. If the scaling factors are the same, the transformation is uniform.

The new coordinates (x', y') of the point after scaling can be calculated as:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} c_x & 0 \\ 0 & c_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

We can express the new coordinates (x', y') of the point after scaling as:

$$x' = c_x \cdot x$$

$$y' = c_y \cdot y$$

Rotation Rotation is a transformation that changes the orientation of an object by rotating it around a specific point, usually the origin.

The rotation transformation matrix R_θ for a rotation by an angle θ is defined as:

$$R_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

where θ is the angle of rotation in radians.

The rotation transformation can be applied to a point (x, y) in the image plane to obtain the new coordinates (x', y') after rotation:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

The new coordinates (x', y') of the point after rotation can be calculated as:

$$x' = x \cdot \cos\theta - y \cdot \sin\theta$$

$$y' = x \cdot \sin\theta + y \cdot \cos\theta$$

Translation Translation is a transformation that moves an object from one location to another without changing its shape or orientation, It is represented as a **shift** in the coordinates of the points in the image, equivalent to changing the origin of the coordinate system.

We basically want to move the point (x, y) to a new location (x', y') by adding a translation vector (x_0, y_0):

$$x' = x + x_0$$

$$y' = y + y_0$$

To do this, we can represent the translation transformation using homogeneous coordinates, which allows us to express it as a matrix multiplication. The translation transformation matrix D_{x_0,y_0} can be represented as:

$$D_{x_0,y_0} = \begin{bmatrix} 1 & 0 & x_0 \\ 0 & 1 & y_0 \\ 0 & 0 & 1 \end{bmatrix}$$

The new coordinates (x', y') of the point after translation can be calculated as:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & x_0 \\ 0 & 1 & y_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

where x_0 and y_0 are the translation distances in the x and y directions, respectively.

6.1.2 Camera Matrix

All the transformations we have seen so far can be combined into a single transformation matrix, called the **camera matrix**. This allows us to represent the relationship between the 3D world and the 2D image plane in a compact form, allowing to correctly map points from the 3D world to the 2D image plane.

The camera matrix is typically a combination of 4 parameters: rotation angle, translation vector, and the scaling factor.

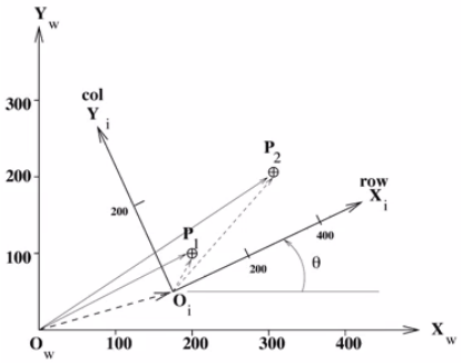


Figure 21: Camera matrix in the 2D case.

From the example, we can see how the point P_1, from the real world, is projected onto the image plane X_i, Y_i as the point P_2. This is done by applying a translation from the real world coordinates to the camera coordinates, then applying a rotation of angle θ and finally applying a scaling factor s to get the final coordinates of the point in the image plane.

Given a point P_j in the real world, we can express the transformation to the image plane as follows:

$$P_j = D_{x_0,y_0} R_\theta S_{s_x,s_y} P_i$$

where P_i is the point in the image plane, D_{x_0,y_0} is the translation matrix, R_θ is the rotation matrix, and S_{s_x,s_y} is the scaling matrix.

Compute the Camera Matrix We can compute the parameters of the camera matrix by using a set of known points, called **control points** in the real world and their corresponding points in the image plane.

Since we have 4 parameters to estimate, we need at least 4 equations to solve for these parameters. Each control point provides us with 2 equations (one for the x-coordinate and one for the y-coordinate), so we need at least 2 control points.

For each control point, we can write the following equations:

$$\begin{bmatrix} x_w \\ y_w \end{bmatrix} = \begin{bmatrix} 1 & 0 & x_0 \\ 0 & 1 & y_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} c_x & 0 & 0 \\ 0 & c_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}$$

where (x_w, y_w) are the coordinates of the point in the real world, (x_i, y_i) are the coordinates of the point in the image plane, (x_0, y_0) are the translation parameters, (c_x, c_y) are the scaling parameters, and θ is the rotation angle.

6.1.3 General Camera Matrix

Given that the three matrices that we are multiplying $(R_\theta, S_{s_x,s_y}, D_{x_0,y_0})$ are all of the same size, we can combine them into a **single camera matrix** C:

$$C = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{bmatrix}$$

Given the camera matrix C, we can compute the transformation from the real world to the image plane and vice versa. For example, given a point $P_j = \begin{bmatrix} x_j \\ y_j \\ 1 \end{bmatrix}$ in the real world, we can compute its projection on the image plane as follows:

$$P_i = CP_j$$

Least Square Approach To obtain the 6 parameters of the camera matrix C, we need at least 3 control points. A good rule of thumb is to select them in a way that they are not collinear or too close to each other, to avoid numerical instability in the estimation of the parameters. But, a more effective way to reduce the error in the estimation of the camera matrix is to use more than 3 control points and solve for the parameters using a least squares approach.

Our goal is to find the set of parameters that minimizes the **error** between the projected points and the actual points in the image plane. The error can be

defined as the sum of the squared distances between the projected points and the actual points in the image plane:

$$E = \sum_{j=1}^{N} (a_{11}x_j + a_{12}y_j + a_{13} - u_j)^2 + (a_{21}x_j + a_{22}y_j + a_{23} - v_j)^2$$

where (u_j, v_j) are the coordinates of the point in the image plane corresponding to the control point $P_j = \begin{bmatrix} x_j & y_j & 1 \end{bmatrix}$ in the real world.

The solution to this optimization problem can be found by computing the partial derivatives of the error function with respect to the parameters of the camera matrix, setting them to zero, and solving the resulting system of equations.

The resulting equation system is:

$$\begin{bmatrix} \sum x_j^2 & \sum x_j y_j & \sum x_j & 0 & 0 & 0 \\ \sum x_j y_j & \sum y_j^2 & \sum y_j & 0 & 0 & 0 \\ \sum x_j & \sum y_j & N & 0 & 0 & 0 \\ 0 & 0 & 0 & \sum x_j^2 & \sum x_j y_j & \sum x_j \\ 0 & 0 & 0 & \sum x_j y_j & \sum y_j^2 & \sum y_j \\ 0 & 0 & 0 & \sum x_j & \sum y_j & N \end{bmatrix} \begin{bmatrix} a_{11} \\ a_{12} \\ a_{13} \\ a_{21} \\ a_{22} \\ a_{23} \end{bmatrix} = \begin{bmatrix} \sum x_j u_j \\ \sum y_j u_j \\ \sum u_j \\ \sum x_j v_j \\ \sum y_j v_j \\ \sum v_j \end{bmatrix}$$

This system of equations can be solved using standard linear algebra techniques, such as Gaussian elimination or singular value decomposition, to find the parameters of the camera matrix C.

6.2 3D Case

What we have seen so far is a simplified version of the camera geometry, where we have only considered the 2D case. This was only applicable to the case where the camera is looking at a flat surface, which is perfectly parallel to the image plane.

In the real world, we usually have a 3D scene and the camera is looking at it from a certain point of view. In this case, we also need to consider the **calibration** of the camera, which is the process of estimating the **intrinsic** parameters of the camera, and the **extrinsic** parameters.

6.2.1 Calibration

As we just said, we need to consider both intrinsic and extrinsic parameters of the camera to correctly model the camera geometry in the 3D case.

Intrinsic set of parameters that describe the internal characteristics of the camera, and are specific to the sensor itself. They include parameters such as the focal length, image distortion, and the size of the pixels.

These parameters are crucial for accurately modeling the 3D geometry of the scene. In fact, since each camera has its own intrinsic parameters, they will affect the outcome of the projection of the 3D points onto the image plane.

Extrinsic set of parameters that describe the position and orientation of the camera in the 3D world. They include parameters such as the rotation and translation of the camera with respect to a reference coordinate system.

Calibration is the process of estimating both intrinsic and extrinsic parameters of the camera, that allow us to correctly map the 3D points in the world to the 2D points in the image plane.

6.2.2 3D representation

In the 3D case, we can represent a point in the world as a 3D vector:

$$P = \begin{bmatrix} X, Y, Z \end{bmatrix}^T$$

where X, Y, Z are the coordinates of the point in the 3D space.

We can also represent the point in homogeneous coordinates as:

$$P = \begin{bmatrix} sX, sY, sZ, s \end{bmatrix}^T$$

where s is a scaling factor, commonly set to 1.

In general, each point has 4 coordinates systems:

- **Model coordinate system**: the coordinate system defined by the 3D model of the scene, with the origin at a specific point in the scene and the axes aligned with the model;
- **World coordinate system**: the real-world coordinate system;
- **Camera 1 coordinate system**: the coordinate system defined by one camera;
- **Camera 2 coordinate system**: the coordinate system defined by another camera;

Since the two cameras can have opposite views of the scene, there is no guarantee that there exists a one-to-one mapping for all the points in the scene. In fact, some points might be visible in one camera and not in the other, due to occlusions or the limited field of view of the cameras. For this kind of points, we cannot find their corresponding points in the 3D space.

6.2.3 3D affine transformation

Similar to the 2D case, we can also define affine transformations in the 3D case. The main difference is that we need to consider the z-coordinate of the points in the 3D space, and the transformation matrices will be of size 4x4 instead of 3x3.

Note

In the following section we will use a slightly different notation from the previous one. In particular, we represent as:

$$^{k}P_i$$

the pixel P_i in the coordinate system of the camera k.

Translation The translation transformation, mapping a point $^{1}P_i$ in the coordinate system of camera 1 to a point $^{2}P_i$ in the coordinate system of camera 2, can be represented as:

$$^{2}P_i = T(t_x, t_y, t_z)\,^{1}P_i$$

where $T(t_x, t_y, t_z)$ is the translation matrix defined as:

$$T(t_x, t_y, t_z) = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where (t_x, t_y, t_z) are the translation distances in the x, y, and z directions, respectively.

Scaling The scaling transformation, mapping a point $^{1}P_i$ in the coordinate system of camera 1 to a point $^{2}P_i$ in the coordinate system of camera 2, can be represented as:

$$^{2}P_i = S(s_x, s_y, s_z)\,^{1}P_i$$

where $S(s_x, s_y, s_z)$ is the scaling matrix defined as:

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where (s_x, s_y, s_z) are the scaling factors in the x, y, and z directions, respectively.

Rotation The rotation transformation needs to consider the three angles of rotation around the x, y, and z axes.

The rotation transformation, mapping a point ${}^1P_i$ in the coordinate system of camera 1 to a point ${}^2P_i$ in the coordinate system of camera 2, can be represented as:

- Rotation around the x-axis:

$$ {}^2P_i = R(X, \theta_x) {}^1P_i $$

where $R(X, \theta_x)$ is the rotation matrix defined as:

$$ R(X, \theta_x) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta_x & -\sin\theta_x & 0 \\ 0 & \sin\theta_x & \cos\theta_x & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

where θ_x is the angle of rotation around the x-axis in radians.

- Rotation around the y-axis:

$$ {}^2P_i = R(Y, \theta_y) {}^1P_i $$

where $R(Y, \theta_y)$ is the rotation matrix defined as:

$$ R(Y, \theta_y) = \begin{bmatrix} \cos\theta_y & 0 & \sin\theta_y & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta_y & 0 & \cos\theta_y & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

where θ_y is the angle of rotation around the y-axis in radians.

- Rotation around the z-axis:

$$ {}^2P_i = R(Z, \theta_z) {}^1P_i $$

where $R(Z, \theta_z)$ is the rotation matrix defined as:

$$ R(Z, \theta_z) = \begin{bmatrix} \cos\theta_z & -\sin\theta_z & 0 & 0 \\ \sin\theta_z & \cos\theta_z & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

where θ_z is the angle of rotation around the z-axis in radians.

6.2.4 General Transformation Matrix

Similar to the 2D case, we can also define a general camera matrix in the 3D case, which combines all the transformations we have seen so far into a single matrix. The general camera matrix C can be represented as:

$$C = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This matrix allows us to represent the transformation from real from camera 1 points ${}^1P_i$ to camera 2 points ${}^2P_i$ as follows:

$${}^2P_i = C {}^1P_i$$

6.2.5 Camera Model

The General Transformation Matrix C is a powerful tool to represent the transformation between different camer coordinate systems, but it doesn't allow to map points from the 3D world to the 2D image plane, since it doesn't consider the intrinsic parameters of the camera.

To do this, we need to define a camera model that combines both the intrinsic and extrinsic parameters of the camera. Given the matrix ${}^I_W C$, we can map points from the world coordinate system ${}^W P$ to the image plane ${}^I P$ as follows:

$${}^I P = {}^I_W C {}^W P$$

where ${}^I_W C$ is the camera matrix that combines both intrinsic and extrinsic parameters of the camera, and ${}^W P$ is the point in the world coordinate system.

Formally, this transformation can be represented as:

$$\begin{bmatrix} s' P_x \\ s' P_y \\ s' \end{bmatrix} = {}^I_W C \begin{bmatrix} {}^W P_x \\ {}^W P_y \\ {}^W P_z \\ 1 \end{bmatrix}$$

6.2.6 Extension of the Camera Model

The previously defined camera model is a simplified version of the real camera model, which is based on the pinhole camera model. In fact, the points in the real world are considered as continuous, while the points in the image plane are discrete, since they are represented as pixels.

We need to apply some additional transformations to the camera model, such as:

- **Roto-Translation**: to account for the rotation and translation of the camera in the 3D world, we apply a roto-translation transformation to the camera model, which allows us to correctly map points from the world coordinate system ${}^{W}P$ to the camera coordinate system ${}^{C}P$.
- **Projection**: to map the 3D points in the camera coordinate system ${}^{C}P$ to the 2D points into an intermediate image plane ${}^{F}P$, we apply a projection transformation.
- **Pixel Scaling**: to account for the size of the pixels in the image plane, we apply a scaling transformation to the camera model, which allows us to correctly map points from the camera coordinate system ${}^{F}P$ (still in mm) to the image plane ${}^{I}P$ (in pixels).

This entire process gives us our final camera model, which allows us to correctly map points from the 3D world to the 2D image plane.

It can be defined as follows:

$$ {}^{I}P = C\,{}^{W}P $$

where C is the final camera matrix that combines all the transformations we have seen so far, including the roto-translation, projection, and pixel scaling transformations.

$$ C = \begin{matrix} c_{11} & c_{12} & c_{13} & c_{14} \\ c_{21} & c_{22} & c_{23} & c_{24} \\ c_{31} & c_{32} & c_{33} & 1 \end{matrix} $$